

1.Problem Analysis

Business context

A lot of what software engineers actually spend their day on isn't writing new features, it's the repetitive stuff around bugs. Someone reports an issue, you have to figure out what they actually mean, go find the part of the codebase it's probably coming from, write a first attempt at a fix, review that fix, write tests for it, document what changed, and then open a pull request so someone can approve it. None of that is hard exactly, it's just a lot of steps, and it pulls engineers away from the harder problems they'd rather be solving.

At the same time, LLMs have gotten good and cheap enough that automating a good chunk of this workflow is actually realistic now, as long as it's done carefully, meaning the system shows its work and never merges anything without a human saying yes. That's basically the idea behind this project: take the everyday bug fixing lifecycle and build a multi agent system around it that produces something real and checkable at the end, not just a suggestion you have to take on faith.

Stakeholders

- Individual developers or small teams who want help triaging and drafting fixes on their own repos, while still being the one who decides what actually gets merged.
- Open source maintainers, who deal with a constant stream of issues and could use something to help pre investigate a report before a human maintainer has to look at it.
- Engineering leads who care about routine maintenance work moving faster, without giving up code review as a step.
- Me, honestly, since I built this and also used it as the actual end user, including running it against my own portfolio repo and merging a real PR it opened.

Problem statement

Most AI coding tools right now are basically one model responding to one prompt in an editor. They're good at generating code, but they don't really model the full lifecycle of an issue: understanding what's being asked, actually looking at the real codebase to ground the answer instead of guessing, reviewing the change, testing it, writing docs for it, and only then handing it to a human to approve.

What felt missing was a system that treats bug fixing as the multi step process it actually is, where each step has an owner, the steps hand off to each other in a way you can actually follow, the whole thing is grounded in the real target repo instead of

general knowledge, and nothing touches the real repo until a human explicitly approves it.

Objectives

- Build a multi agent system where each agent does one specific job in the bug fixing lifecycle, instead of one agent trying to do everything.
- Make the handoffs between agents explicit and reliable, using a shared state object and clear routing logic, so you can actually see how information flows between agents rather than it being buried in one big prompt.
- Ground the system's answers in the real codebase by retrieving actual relevant code, instead of letting the model just guess from general training knowledge.
- Give it memory, so once an issue gets resolved, similar issues in the future can reference what was done before.
- Build in real self correction, where one agent can reject another agent's work and send it back for another attempt, and where an agent can catch and retry its own failure, but always with a limit that escalates to a human instead of looping forever.
- Require a genuine human approval step before anything touches a real repository, and make sure that once approved, the action that happens is real, not simulated.
- Actually deploy it somewhere public, and prove it works by running it against both a well known open source repo and my own repo, including getting a resulting pull request all the way to a real merge.

2.Multi Agent Design

Agent architecture

Instead of building one agent with a giant prompt trying to do everything, I split the work up. Each step in the bug fixing lifecycle gets its own agent with one job, and the whole thing runs as a graph using LangGraph rather than a fixed sequence of function calls. Every agent reads from and writes to one shared state object, and the graph itself decides what runs next based on conditions, not a hard coded order. That's what lets it branch, loop back when something gets rejected, and run two agents at the same time when it makes sense, instead of forcing everything through one straight path.

There are six agents in the core pipeline: Requirements Analysis, Bug Investigation, Coding Assistant, Code Reviewer, Testing Agent, and Documentation Writer. Around them sits an orchestration layer that handles the routing, a retrieval layer that grounds the agents in the real target codebase, a memory layer that remembers resolved issues

across runs, and an actions layer that turns an approved fix into an actual GitHub pull request.

Roles of each agent

- Requirements Analysis reads the raw GitHub issue and turns it into something structured, basically what kind of issue it is, a short summary, a list of acceptance criteria, and a rough complexity guess. Everything after this depends on it getting this part right.
- Bug Investigation only kicks in if the issue was classified as a bug. It pulls the most relevant real code from the target repo using retrieval, and reasons about the likely cause using that actual code instead of just guessing from general training knowledge.
- Coding Assistant takes what Investigation found and proposes an actual fix, still grounded in the real retrieved code, along with a risk level and a flag for whether it thinks a human should look at it before anything happens.
- Code Reviewer checks the proposed fix and either approves it or sends it back with feedback. If it's rejected, it goes back to Coding Assistant for another try, up to a limited number of attempts, after which the system stops looping and just escalates to a human instead of getting stuck.
- Testing Agent actually writes a self contained pytest file and runs it for real in a sandboxed process, it's not just generating text that looks like a test. If the test itself is broken, it takes that failure and tries again.
- Documentation Writer writes a changelog entry, some inline code comments, and a short readme style summary of what changed.

Agent interaction and handoff flow

It starts by fetching the issue from GitHub, then goes into Requirements Analysis. From there the orchestrator checks what type of issue it is. If it's a bug, it goes to Bug Investigation first, otherwise it skips straight to Coding Assistant. Coding Assistant always hands off to Code Reviewer next.

This is where the first real loop shows up. If the reviewer rejects the fix, it goes back to Coding Assistant with the feedback attached and tries again. This can happen a couple of times before the system just stops looping and treats it as resolved anyway, flagging it for a human instead of trying forever.

Once review is actually settled, whether it got approved or just hit the revision limit, the graph splits into two at once, Testing Agent and Documentation Writer run in parallel since neither one needs the other's output. Once both are done, the whole pipeline just stops and waits. Nothing after that happens on its own. A person has to

actually look at it and click approve. Only then does it hand off to the part that forks the repo, makes a branch, commits the fix, and opens a real pull request.

Tool integration overview

- The Groq API is what every single agent actually calls to think, using a fast, free tier model.
- The GitHub REST API gets used twice, once early on to pull the original issue, and again later, after a human approves, to fork the repo, create a branch, commit the fix, and open the PR.
- ChromaDB is the vector database behind two separate things, retrieval over the target codebase and long term memory of issues that were already resolved.
- Sentence Transformers runs locally to generate the embeddings ChromaDB stores and searches, so there's no extra embedding API needed.
- pytest is the actual test runner, and it runs in a real subprocess sandbox, so what the Testing Agent reports is a genuine pass or fail, not just generated text pretending to be a result.

3.Implementation

Minimum 5 specialized AI agents

There are six agents total, and each one only does one thing. Requirements Analysis just classifies and structures the issue. Bug Investigation just investigates, using retrieved code, not guesses. Coding Assistant just proposes a fix. Code Reviewer just judges the fix. Testing Agent just writes and runs tests. Documentation Writer just writes the docs. I kept them narrow on purpose, because the moment one agent starts doing two jobs, the handoffs between agents stop meaning anything.

Minimum 5 tools/APIs

Five tools are actually wired in and doing real work, not just name dropped:

- Groq API, which every single agent calls to think
- GitHub REST API, used to pull the issue in and, later, to fork, branch, commit, and open the pull request
- ChromaDB, the vector database behind both retrieval and long term memory
- Sentence Transformers, running locally to generate the embeddings for ChromaDB

- pytest, run in a real subprocess, so test results are actual pass or fail, not just text that looks like one

Agent handoffs

Agents don't call each other directly. Everything passes through one shared, typed state object, and the orchestrator decides what runs next based on that state, using conditional routing instead of a fixed sequence. That's what lets it branch, like skipping investigation entirely for a non bug issue, loop back from Code Reviewer to Coding Assistant when a fix gets rejected, and run Testing Agent and Documentation Writer at the same time once review is done.

Memory/context management

Two layers here. Within a single run, the shared state carries everything forward, the issue, the requirements, the investigation, the fix, the review, the test results, all of it, so nothing downstream has to re ask or re fetch anything. Across runs, approved fixes get embedded and saved into ChromaDB, and before investigating a new bug, the system checks that memory for anything similar it's already resolved. So it actually gets more useful the more you run it, instead of starting from zero every time.

Structured outputs

Every agent has to return strict JSON, no loose free text getting parsed on a hope. Free tier models occasionally return broken or even totally empty responses, so there's a shared parser that tries a few layers of repair before giving up, stripping markdown, fixing trailing commas, fixing bad backslashes in generated code, and worst case, just asking the model to fix its own output. On top of that, every API call is wrapped so it retries from scratch if the response comes back empty, which is what actually fixed the issue once I traced it back to the model quietly burning its whole token budget on invisible reasoning before writing anything real.

Human approval

Nothing touches a real repository without a person saying yes first. Once a fix passes review, and testing and docs are done, the whole pipeline just stops and shows an approval gate. Only after someone clicks approve does it move on to forking the repo, creating a branch, committing the fix, and opening a real pull request. Not simulated, genuinely real, I confirmed this by running it against a well known open source project and against my own repo, and actually merging the pull request it opened.

4.Advanced Features

Planning and reasoning

The orchestrator does real planning, it's not just a straight line of calls. It decides whether to even run Bug Investigation based on how Requirements Analysis classified the issue, and it decides how many times to send a rejected fix back to Coding Assistant before giving up and handing it to a human. That routing is what actually plans the path through the agents instead of running the same sequence no matter what.

RAG / Knowledge retrieval

This is the part I'm most happy with, honestly. Instead of the agents guessing from general training knowledge, the system clones and indexes whatever repo gets entered, on demand, into its own vector collection. Bug Investigation and Coding Assistant both pull real chunks of that actual code before they reason about anything. It's not hard coded to one demo repo either, I tested this by pointing it at a completely different, personal project and watching it clone, index, and reason over that codebase correctly.

Long term memory

Every time a fix gets approved, it gets embedded and saved separately. Before investigating a new bug, the system checks that memory for similar issues it's already solved and pulls those in as context. So it's not just retrieving code, it's also remembering its own past work.

Reflection/self review

There are two of these, not one. Code Reviewer can reject a fix and kick it back to Coding Assistant with feedback, up to a few times, before the system stops looping and escalates to a human instead. Testing Agent has its own separate loop too, if the test it writes fails to even run because of a mistake in the test code itself, it takes that failure and tries generating the test again. I actually saw this recover from a real failure while testing, not just as a theoretical feature.

Parallel agent execution

Once a fix clears review, Testing Agent and Documentation Writer run from that same point at the same time, since neither needs the other's output. This was actually a bug I had to fix partway through, documentation was originally wired to run every time review ran at all, even on rejected attempts, so it was writing docs for fixes that weren't even approved yet. Fixing that also meant each run burned a lot less of the rate limit.

Error handling and logging

This one got way more real world testing than I expected. Every agent falls back to sensible defaults if the model skips a field it was supposed to return. JSON parsing has several layers of automatic repair before it gives up completely. API calls retry on their own if the model comes back empty, which turned out to be an actual traceable bug, not random bad luck. Network and rate limit errors get caught and shown clearly instead of crashing or failing

silently. The run button even disables itself while a pipeline is active, because I hit a real bug where clicking it twice fired two runs at once and burned through the rate limit twice as fast.

Multi modal inputs

Not built. This would mean handling something like a screenshot of a bug alongside the text. I made the call to skip it and put that effort into making the other seven features actually solid instead.

Session persistence

Partially there, being honest. ChromaDB does persist across restarts, so the RAG index and the long term memory both survive. What's missing is saving and resuming a pipeline run that's still in progress, if the app restarts mid run, that one run is lost, even though everything it had already learned and stored up to that point is not.